

Deep Learning: Training DNNs – Part I

1. What is Optimization?

When we train a neural network, we want to find the best possible values for all the weights (W) and biases (b) so that our model makes the smallest possible error on the training data. This process is called optimization.

For simple models like linear regression, we can directly calculate the optimal answer using a closed-form solution — a single mathematical formula gives us the answer. But for deep neural networks, the loss function is far too complicated to solve directly. So we turn to an iterative method called Gradient Descent combined with Backpropagation.

The Core Goal

We want to find the weights W^* and biases b^* that minimize the training loss L . Written formally:

$$W^*, b^* = \operatorname{argmin} L(f(\cdot; W, b); D)$$

This just means: find the W and b values that give us the lowest possible loss over the entire dataset D .

Think of the loss as a hilly landscape — very high-dimensional. Our job is to roll a ball to the lowest valley on that landscape. That's what gradient descent does, step by step.

2. Batch Gradient Descent

In Batch Gradient Descent, we compute the gradient of the loss using the ENTIRE dataset before taking a single update step. The algorithm looks like:

```
for i in range(nb_epochs):  
     $\nabla L_w$  = evaluate_gradient(L, D, w)    # gradient over full dataset D  
    w = w -  $\eta$  *  $\nabla L_w$                   # update the weights
```

The weight update formula is:

$$w \leftarrow w - \eta \cdot \nabla_w L(w)$$

η (eta) is the learning rate — it controls how big a step we take.

Key property: Batch GD is guaranteed to converge to the global minimum for convex loss functions, and to a local minimum for non-convex ones (like neural networks).

The downside is that for large datasets, computing the gradient over ALL examples for every single weight update is extremely slow and memory-heavy.

3. Stochastic Gradient Descent (SGD)

SGD goes to the opposite extreme — instead of using the whole dataset, it updates the weights after looking at just ONE training example at a time.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} L(\mathbf{w}, \mathbf{x}_i, y_i)$$

Here $(\mathbf{x}_i, y_i)$ is a single training example chosen at random.

The algorithm:

```
for i in range(nb_epochs):  
    np.random.shuffle(D)          # shuffle the data  
    for x_i in D:  
         $\nabla L_{\mathbf{w}}$  = evaluate_gradient(L, x_i, w)  
        w = w -  $\eta$  *  $\nabla L_{\mathbf{w}}$ 
```

Why SGD is Useful

SGD computes a noisy estimate of the gradient. This noise is actually helpful — even if the true gradient is zero at a saddle point, the noisy approximation will point in some random direction, allowing the optimizer to escape that stuck position. SGD also starts learning immediately and requires only one sample in memory at a time.

The Problem with SGD

Because it uses only one example at a time, the updates fluctuate a lot. The loss function bounces up and down, making convergence messy. The optimizer often overshoots the minimum.

However, there is good news: if we slowly reduce the learning rate over time, SGD can converge to a solution similar to what Batch GD would find.

4. Mini-batch Gradient Descent

Mini-batch GD is the method that people actually use in practice. Instead of one example or all examples, it processes a small batch of n examples at a time. This is the sweet spot — stable like batch GD but efficient like SGD.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} L(\mathbf{w}, \mathbf{x}_{i:i+n}, y_{i:i+n})$$

The subscript $i:i+n$ means we take examples i through $i+n$ as a batch.

The algorithm:

```
for i in range(nb_epochs):
```

```
np.random.shuffle(D)
for batch in get_batches(D, batch_size=128):
    ∇L_w = evaluate_gradient(L, batch, w)
    w = w - η * ∇L_w
```

Why Mini-batch?

Mini-batch reduces the variance of the parameter updates compared to SGD, leading to more stable convergence. It also allows the use of highly optimized matrix operations on GPUs which speeds up computation massively.

Common mini-batch sizes range from 32 to 1024 depending on the task and GPU memory. When people in deep learning say 'SGD,' they almost always mean mini-batch gradient descent — even though the name is technically incorrect.

5. Challenges in Training

Challenge 1: Choosing the Right Learning Rate

The learning rate η is the most critical hyperparameter. If it is too large, the updates overshoot the minimum and the training diverges. If it is too small, training will take forever. The ideal learning rate also changes over time — you want bigger steps early on and smaller steps as you approach the optimum.

The general rule for the learning rate and convergence is:

$$\eta < 2 \cdot \eta_{\min}$$

The learning rate must be smaller than twice the smallest optimal learning rate for any component. If this condition is violated, training may diverge.

Challenge 2: Same Learning Rate for All Parameters

Standard SGD applies the same learning rate to every single weight in the network. But in reality, different parameters need different-sized updates. Some dimensions of the loss surface may be steep, others very shallow. One global learning rate cannot handle all of them optimally.

For a multivariate quadratic function of the form:

$$f(\mathbf{x}) = (1/2) \cdot \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{x}^T \mathbf{b} + c$$

If $\mathbf{A}$ is diagonal (as simplified), the optimal learning rate is different for each component. This is why adaptive learning rate methods were invented.

Challenge 3: Saddle Points and Local Minima

The loss surface of a deep network is a very high-dimensional function. It has local minima (small valleys that are not the global minimum) and saddle points (flat regions that look like a minimum from one direction but are actually a maximum from another

direction). Basic gradient descent can get trapped at these points since the gradient is zero there.

The general SGD update at each step is:

$$w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$$

6. Learning Rate Schedules

Since we want the learning rate to start high and decrease over time, we use learning rate schedules. These are predefined rules for how the learning rate changes at each epoch or step. The general idea is: use a high learning rate early when you are far from the minimum, and reduce it as you get closer.

Step Decay

The simplest schedule. Reduce the learning rate by a fixed factor after every fixed number of epochs. For example: after every 30 epochs, multiply the current learning rate by 0.1.

$$\eta^* = 0.1 \cdot \eta \quad (\text{e.g., every 30 epochs})$$

This produces a characteristic staircase-shaped loss curve with visible 'jumps' when the learning rate drops. The downside is that it introduces annoying hyperparameters — you have to decide when to reduce the learning rate and by how much.

Cosine Decay

A smoother approach. The learning rate follows a cosine curve, starting at its initial value and gradually reducing toward zero. The formula is:

$$\eta_t = (1/2) \cdot \eta_0 \cdot (1 + \cos(t \cdot \pi / T))$$

t = current step, T = total steps, η_0 = initial learning rate

Cosine decay requires fewer hyperparameter decisions and tends to work well in practice. One known issue: training longer generally helps, but finding the right initial learning rate η_0 is still tricky.

Linear Decay

The learning rate decreases in a straight line from start to finish:

$$\eta_t = \eta_0 \cdot (1 - t/T)$$

Exponential Decay

The learning rate decreases exponentially:

$$\eta_t = \eta_0 \cdot (1 - \alpha/100)^t$$

α controls how fast the decay happens.

Constant Learning Rate

$$\eta_t = \eta_0$$

No change at all. Surprisingly this works fine for quick prototyping and experimenting with ideas. If you want to squeeze out that last 1–2% of performance, you would need a proper schedule — but for initial experiments, constant is perfectly reasonable.

7. SGD with Momentum

Plain SGD tends to jitter a lot, especially on loss surfaces that are steep in one direction and shallow in another. Momentum addresses this by giving the optimizer a kind of 'memory' — it accumulates a velocity based on past gradients and uses that to update the weights, rather than purely following the current gradient.

Intuition

Imagine a ball rolling downhill. Even if the slope temporarily flattens out or becomes a saddle point, the ball keeps rolling because of its built-up momentum. The optimizer behaves the same way — even when the gradient is zero (saddle point), the accumulated velocity keeps it moving. This helps escape saddle points and local minima.

The Formulas

Compare SGD and SGD+Momentum side by side:

Plain SGD:

$$w_{t+1} = w_t - \eta \cdot \nabla_w J(w)$$

SGD + Momentum:

$$\begin{aligned} v_0 &= 0 \\ v_{t+1} &= \rho \cdot v_t + \nabla_w J(w) \\ w_{t+1} &= w_t - \eta \cdot v_{t+1} \end{aligned}$$

v is the velocity (momentum). ρ (rho) is the friction coefficient, typically 0.9 or 0.99.

The code implementation:

```
v = 0
for i in range(num_iters):
    dw = grad(J, W, x, y)
    v = rho * v + dw          # accumulate velocity
    w -= eta * v              # update weights using velocity
```

Why Momentum Helps

Momentum reduces the jitter in ravine-like loss surfaces where one direction is very steep and another is very shallow. Updates become more smoothed out because the velocity is an exponential moving average over past gradients. And crucially, optimization proceeds even at local minima or saddle points where the gradient alone would be zero — the accumulated velocity carries it through.

Nesterov Momentum (NAG)

Nesterov Accelerated Gradient is an improvement over standard momentum. The key difference: instead of computing the gradient at the current position and then taking a step, Nesterov first takes a 'look-ahead' step in the direction of the current velocity, and then computes the gradient from that future position.

This makes the optimizer more responsive — it can correct itself faster, especially for large values of ρ .

```
v = 0
for i in range(num_iters):
    dw = grad(J, W + rho*v, x, y)    # gradient from look-ahead position
    v = rho * v + dw
    w -= eta * v
```

8. Adaptive Learning Rates

8.1 AdaGrad (Duchi et al., 2011 – JMLR)

AdaGrad introduced the idea of per-parameter adaptive learning rates. Instead of applying the same learning rate to all parameters, it scales the learning rate individually for each parameter based on how large the gradients for that parameter have been so far.

Parameters that receive large, frequent gradient updates get a smaller effective learning rate. Parameters that receive small or infrequent updates get a larger effective learning rate. This makes AdaGrad well-suited for sparse data (e.g., word embeddings in NLP).

The algorithm:

```
grad_sq = 0
for i in range(max_iters):
    dw = grad(J, w, x, y)
    grad_sq += dw @ dw                # element-wise square,
    accumulated                       # accumulated
    w -= eta * dw / (sqrt(grad_sq) + epsilon)  # adaptive update
```

$$\mathbf{w} \leftarrow \mathbf{w} - \left[\eta / (\sqrt{G} + \epsilon) \right] \cdot \nabla_{\mathbf{w}} J(\mathbf{w})$$

G is the accumulated sum of squared gradients. ϵ is a small constant (like $1e-8$) to avoid division by zero. $\odot$ denotes element-wise multiplication.

The Problem with AdaGrad

Since G only ever grows (we keep adding gradient squares), the effective learning rate keeps shrinking. If you run AdaGrad long enough, the learning rate eventually approaches zero and learning effectively stops. This is a major limitation.

8.2 RMSProp

RMSProp (Root Mean Square Propagation) fixes AdaGrad's dying learning rate problem. Instead of accumulating ALL past squared gradients (which grows forever), RMSProp uses a leaky/decaying version — a rolling average that gradually forgets old gradients using a friction coefficient ρ .

The algorithm:

```
grad_sq = 0
for i in range(max_iters):
    dw = grad(J, w, x, y)
    grad_sq =  $\rho$  * grad_sq + (1- $\rho$ ) * dw  $\odot$  dw      # leaky accumulation
    w -=  $\eta$  * dw / (sqrt(grad_sq) +  $\epsilon$ )
```

$$G_t = \rho \cdot G_{t-1} + (1 - \rho) \cdot (\nabla_w J)^2$$
$$w \leftarrow w - \left[\eta / (\sqrt{G_t} + \epsilon) \right] \cdot \nabla_w J(w)$$

The key change from AdaGrad: G is now an exponentially decaying average, so it does not grow to infinity. Old gradients fade away, preventing the learning rate from dying out.

8.3 Adam – Adaptive Moment Estimation

Adam is the most widely used optimizer in modern deep learning. It combines the two best ideas we have seen: momentum (like SGD+Momentum) and adaptive learning rates (like RMSProp). It maintains two moving averages: $m1$ (first moment — the mean of the gradients, like momentum) and $m2$ (second moment — the mean of the squared gradients, like RMSProp).

Adam = RMSProp + Momentum

The algorithm:

```
m1 = 0      # first moment (momentum)
m2 = 0      # second moment (adaptive scaling)
for i in range(max_iters):
    dw = grad(J, w, x, y)
    m1 =  $\beta_1$  * m1 + (1 -  $\beta_1$ ) * dw      # update first moment
    m2 =  $\beta_2$  * m2 + (1 -  $\beta_2$ ) * dw * dw  # update second moment
    w -=  $\eta$  * m1 / (sqrt(m2) +  $\epsilon$ )        # adaptive update
```

The update formulas:

```
m1_{t+1} = \beta_1 \cdot m1_t + (1 - \beta_1) \cdot \nabla_w J(w)
m2_{t+1} = \beta_2 \cdot m2_t + (1 - \beta_2) \cdot (\nabla_w J(w))^2
w \leftarrow w - \eta \cdot m1 / (\text{sqrt}(m2) + \epsilon)
```

Bias Correction in Adam

There is one subtlety: both $m1$ and $m2$ are initialized to zero. At the very beginning of training, this causes both estimates to be biased toward zero (artificially small). Adam corrects for this bias using:

```
m1_corrected = m1 / (1 - \beta_1^t)
m2_corrected = m2 / (1 - \beta_2^t)
```

As training progresses (t grows), the bias correction factor approaches 1.0, so it only matters in the early steps.

Adam works very well in practice, mostly with a fixed standard set of hyperparameter values. Typical defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$. It is the go-to optimizer for most deep learning tasks.

9. Other Optimizer Variants

Beyond Adam, several other variants have been proposed that build on these ideas in various ways:

AMSGrad

Addresses a theoretical issue with Adam where the adaptive learning rate can be non-monotonically decreasing. AMSGrad uses the maximum of past $m2$ values to guarantee convergence.

Nadam

Adam + Nesterov momentum. Replaces standard momentum in Adam with the look-ahead Nesterov version for slightly faster correction.

AdaMax

A variant of Adam that uses the infinity norm instead of the L2 norm in the denominator, which can be more stable in some cases.

AdaDelta

Another fix for AdaGrad's decaying learning rate, similar in spirit to RMSProp. It does not even require setting an initial learning rate.

10. Summary

Training a deep neural network is fundamentally an optimization problem. We use gradient descent to iteratively move the weights in a direction that reduces the loss. Here is how the story builds up:

Batch GD is mathematically clean but impractically slow. SGD is fast but noisy and unstable. Mini-batch GD combines the best of both and is what we always use in practice.

The learning rate is the single most important hyperparameter. It must be chosen carefully — not too large (diverges), not too small (too slow). Using a learning rate schedule that decays over time is standard practice. The constraint for convergence is that η must be less than $2 \cdot \eta_{\min}$.

Momentum helps the optimizer move smoothly through the loss surface and escape saddle points and local minima by accumulating velocity from past gradients (ρ typically 0.9–0.99). Nesterov momentum improves on this with a look-ahead correction.

Adaptive methods go further by giving each parameter its own effective learning rate. AdaGrad was the first but its learning rate dies out. RMSProp fixes this with a leaky gradient accumulation. Adam combines both momentum and adaptive rates with bias correction, and works extremely well with default settings.

For most deep learning tasks today, Adam is the default starting point.